

Consistency Reasoning for Removing Conflicts and Clauses

Steven Prestwich

Cork Constraint Computation Centre
Department of Computer Science, University College, Cork, Ireland
s.prestwich@cs.ucc.ie

Abstract. This paper describes a consistency-based method for removing conflicts from (binary or non-binary) CSPs and clauses from SAT problems. The more powerful the consistency, the more conflicts can be removed. Removing conflicts creates “pseudo-solutions” which have the property of being transformable into actual solutions in polynomial time. Increasing the solution density of the problem may make it easier to solve, especially by local search. Using only node consistency the method rediscovers some known SAT modelling techniques. In experiments both positive and negative results on SAT benchmarks are reported.

1 Introduction

It is useful to be able to transform a constraint satisfaction problem (CSP) or a Boolean satisfiability (SAT) problem, to make it easier to solve by a class of search algorithms. Research has been done on the automatic addition of implied constraints [4] and on Boolean formula simplification techniques [8]. Another recent approach [7] is to increase the *solution density* of a SAT problem, where solution density is defined as the number of solutions divided by the number of possible assignments. This may improve local search performance, which seems to depend largely (though not solely) on solution density [3, 9–11].

This paper describes a new technique for increasing the solution density of CSPs and SAT problems, by retracting certain conflicts and thus creating additional *pseudo-solutions*. It is designed so that any pseudo-solution can be transformed into a solution in a polynomial time post-processing phase. The creation of pseudo-solutions has been shown to greatly improve local search performance on some problems [11]. Our method is based on consistency processing, and the more powerful the consistency the more conflicts or clauses can be removed.

The paper is organised as follow. Section 2 describes and proves the correctness of the technique on CSPs. Section 3 adapts the technique to SAT problems and shows that it generalises some known SAT modelling techniques. Section 4 evaluates an automated clause retraction system on SAT benchmarks. Section 5 discusses related work. Section 6 concludes the paper. Familiarity with Constraint Programming concepts is assumed.

2 Retractable conflicts

A constraint can be defined extensionally in terms of *conflicts*, each being a forbidden set of literals (or *assignment*) for the constraint variables. We will find it convenient to denote a conflict by an assignment $\{v \leftarrow a, w \leftarrow b, x \leftarrow c, \dots\}$ where $v, w, x, \dots$ are distinct finite domain variables and $a, b, c, \dots$ are domain values; this assignment is forbidden by the conflict. A CSP solution is an assignment for all CSP variables that satisfies each constraint. A conflict C is *violated* by a total assignment S if $C \subseteq S$, otherwise it is *satisfied* by S . We define $\Theta(C)$ to represent the constraint that each of the literals in C must be true. So whereas a conflict $C = \{v \leftarrow a, \dots\}$ forbids its literals from being all true, $\Theta(C)$ asserts that each C literal must be true. This corresponds to a set of unary constraints forbidding all other literals involving C variables: $\Theta(C) = \{(v \leftarrow b) \mid (v \leftarrow a) \in C, b \in \text{dom}(v) \setminus \{a\}\}$. We will sometimes focus on a particular literal in a conflict, and use the symbol Δ to denote its remaining literals. For example we may write a conflict $\{v \leftarrow a, w \leftarrow b, x \leftarrow c\}$ as $\{w \leftarrow b\} \cup \Delta$ where $\Delta = \{v \leftarrow a, x \leftarrow c\}$.

2.1 Full retractability

We would like to find a conflict C that can be removed from P in such a way that any solution to $P \setminus \{C\}$ can be transformed into a solution to P . First a definition.

Definition 1. A literal $v \leftarrow a'$ is a **repair** for a conflict $C = \{v \leftarrow a\} \cup \Delta$ if any solution to $P \setminus \{C\}$ in which $v \leftarrow a$ holds can be transformed to a solution to P by reassigning v to a' . We then say that C is **fully retractable** from P .

Our main result is:

Theorem 1. Let P be a CSP and $C = \{v \leftarrow a\} \cup \Delta$ one of its conflicts. If for any $P \setminus \{C\}$ conflict $C' = \{v \leftarrow a'\} \cup \Delta'$ ($a \neq a'$) each problem $P_{\Delta\Delta'C} = \Theta(\Delta) \cup \Theta(\Delta') \cup P \setminus \{C\}$ is unsatisfiable then C is fully retractable from P with repair $v \leftarrow a'$.

Proof. For any solution S to $P \setminus \{C\}$ there are two cases. (1) If S does not violate C then S is also a solution to P . (2) If S does violate C then S contains $v \leftarrow a$ and all the Δ literals. Reassigning to $v \leftarrow a'$ satisfies C , but we must show that this reassignment does not violate any other conflict. (2a) Any conflict not containing v is unaffected and therefore still satisfied. (2b) Any conflict containing $v \leftarrow a''$, where $a'' \notin \{a, a'\}$, is still satisfied because $\{v \leftarrow a''\} \not\subseteq S$. (2c) To show that a conflict C' containing $v \leftarrow a'$ is still satisfied we must show that $\Delta' \not\subseteq S$. If $P_{\Delta\Delta'C}$ is unsatisfiable then at least one of its conflicts must be violated under S . No $P \setminus \{C\}$ conflict is violated because S is a solution to this problem. By hypothesis S violates C , so $\Delta \subseteq S$ and $\Theta(\Delta)$ is satisfied. Therefore $\Theta(\Delta')$ is violated, and at least one Δ' literal is false in S . $\square$

The following result shows that it is safe to apply conflict retraction:

Theorem 2. Removing a fully retractable conflict from a CSP preserves satisfiability.

Proof. Any solution to P is also a solution to $P \setminus \{C\}$, therefore if P is satisfiable then so is $P \setminus \{C\}$. Conversely, we have shown that any solution to $P \setminus \{C\}$ can be transformed into a solution to P , therefore if $P \setminus \{C\}$ is satisfiable then so is P . $\square$

2.2 Local retractability

For a k -ary conflict C and domains of size d there are $k(d - 1)$ candidate repairs, and in the worst case we may need to test all of them before rejecting C as retractable. Each such test requires showing the unsatisfiability of a $P_{\Delta\Delta'C}$ CSP, which is of course NP-hard. Full retractability is therefore unlikely to be applied in practice.

Fortunately we can use tractable, stronger sufficient conditions for unsatisfiability based on local reasoning. A simple condition is that a problem is not *node consistent*. In fact, assuming that P was already node consistent and contained no unary constraints, we need only check that $\Theta(\Delta) \cup \Theta(\Delta')$ is not node consistent because it contains conflicting literals for a variable. We then say that C is *NC-retractable* (NC denoting node consistency). A weaker condition is the [generalised] arc inconsistency of $P_{\Delta\Delta'C}$, and we then say that C is *[G]AC-retractable*. We could also use higher forms of consistency, global constraints, shaving, or even limited backtrack search.

2.3 Multiple retractions

Retraction can be applied again to $P \setminus \{C\}$ to generate another model, and so on until no further retractions are possible. The output of a series of retractions is a CSP with fewer conflicts, plus a list of instructions on how to reassign certain variables (which must be applied in reverse order). For example consider a problem P containing a retractable conflict

$$C = \{v \leftarrow a, w \leftarrow b, x \leftarrow c\}$$

for which $v \leftarrow a'$ is a repair. The method would generate the model $P \setminus \{C\}$ plus an instruction of the form:

$$\text{if } v \leftarrow a, w \leftarrow b \text{ and } x \leftarrow c \text{ then reassign } v \leftarrow a'$$

which can be used to generate a solution to P from a solution to $P \setminus \{C\}$.

However, we must ensure that no repair is reversed by a later repair, otherwise the correctness of the repaired solution is not guaranteed. A sufficient condition for this is that each variable domain is (arbitrarily) totally ordered, and that a repair that reassigns from $v \leftarrow a$ to $v \leftarrow a'$ is only allowed if $a' \prec_v a$ where $\prec_v$ denotes the ordering on $\text{dom}(v)$. These orderings can be determined before retraction begins, or built up during retraction.

2.4 An example

Consider a CSP with variables a, b, c each with domain $\{1, 2\}$ and conflicts

$$C_1 = \{a \leftarrow 1, b \leftarrow 1\}$$

$$C_2 = \{a \leftarrow 2, c \leftarrow 2\}$$

$$C_3 = \{b \leftarrow 1, c \leftarrow 2\}$$

$$C_4 = \{a \leftarrow 1, c \leftarrow 1\}$$

Conflict C_4 is NC-retractable with repair $a \leftarrow 2$, because $a \leftarrow 2$ occurs only in C_2 and the literals $\Delta = c \leftarrow 1$ and $\Delta' = c \leftarrow 2$ are node-inconsistent. This retraction

introduces a pseudo-solution $\{a \leftarrow 1, b \leftarrow 2, c \leftarrow 1\}$ which can be repaired by the reassignment $a \leftarrow 2$, giving the true solution $\{a \leftarrow 2, b \leftarrow 2, c \leftarrow 1\}$.

Conflict C_1 is *not* NC-retractable with repair $a \leftarrow 2$, because $a \leftarrow 2$ occurs in C_2 and $\Delta = b \leftarrow 1$ is not node-inconsistent with $\Delta' = c \leftarrow 2$. However, C_1 is AC-retractable with repair $a \leftarrow 2$ because C_3 forbids $\Theta(\Delta) \cup \Theta(\Delta')$ so $P_{\Delta\Delta'C_1}$ is arc-inconsistent. This retraction introduces no pseudo-solutions, but if we apply it after retracting C_4 then it introduces pseudo-solution $\{a \leftarrow 1, b \leftarrow 1, c \leftarrow 1\}$ which can be repaired to the true solution $\{a \leftarrow 2, b \leftarrow 1, c \leftarrow 1\}$.

3 Retractable clauses

Any SAT problem can be viewed as a CSP with binary domains and non-binary constraints via the *non-binary encoding* described in [14]. For example a clause $a \vee b \vee \bar{c}$ corresponds to the conflict $\{a \leftarrow F, b \leftarrow F, c \leftarrow T\}$. A repair $v \leftarrow T$ (or $v \leftarrow F$) is a literal $\bar{v}$ (or v). A repair $\bar{\ell}$ for a clause $C = \ell \vee \Delta$ (where Δ is a disjunction of 0, 1 or more literals) is one satisfying the following condition: for any clause $\bar{\ell} \vee \Delta'$, the problem $P_{\Delta\Delta'C}$ is unsatisfiable. In SAT terms $P_{\Delta\Delta'C}$ is P minus clause C , with each literal in $\Delta \vee \Delta'$ added as a unit clause. A repair is now a literal.

The simplest method for detecting unsatisfiability is to check whether Δ' contains at least one negated Δ literal. This is a restricted form of resolution in which only unit clauses from Δ and Δ' are resolved, and corresponds to the node inconsistency method of Section 2. A more powerful method would be to apply unit propagation, or a width-restricted resolution algorithm such as those used for preprocessing in some SAT solvers, or (as noted for the CSP case) by restricted backtrack search.

3.1 Encoded constraint problems

As an application of clause retraction, we show that it rediscovers a well-known SAT technique which often improves local search performance. The most natural and widely-used CSP encoding into SAT is the *direct encoding* [14]. A SAT variable x_{vi} is defined as true if and only if the CSP variable v has the domain value i assigned to it, where $i \in \text{dom}(v) = \{1, \dots, d\}$. The direct encoding consists of three sets of clauses. Each CSP variable must take at least one domain value, expressed by *at-least-one* clauses:

$$x_{v1} \vee x_{v2} \vee \dots \vee x_{vd}$$

No CSP variable can take more than one domain value, expressed by *at-most-one* clauses:

$$\bar{x}_{vi} \vee \bar{x}_{vj}$$

Conflicts are enumerated by *conflict* clauses:

$$\bar{x}_{vi} \vee \bar{x}_{wj}$$

It is well-known that local search often performs better on a direct CSP encoding if the at-most-one clauses are removed [10, 13]. It is easy to show that these are NC-retractable. Consider an at-most-one clause $C = \bar{x}_{vi} \vee \bar{x}_{vj}$. We must show that x_{vi} is

a repair for C . It occurs only in the at-least-one clause $x_{v1} \vee \dots \vee x_{vd}$. But this clause also contains the literal x_{vj} which is the negation of the literal $\bar{x}_{vj}$ from C . Therefore x_{vi} is a repair of C , which is NC-retractable. This applies to all at-least-one clauses.

No conflict clause has a repair because no at-least-one clause contains two literals x_{vi} and x_{vj} . Similarly, no at-least-one clause C has a repair because no conflict clause contains a negated literal from C . However, under a different SAT-encoding the at-least-one clauses become retractable instead of the at-most-one clauses. Consider an example from [5] of a colouring problem with two adjacent vertices x, y and three colours R, G, B . Instead of conflict clauses such as $\bar{x}_{vR} \vee \bar{y}_{wR}$ we may write clauses of the form $x_{vB} \vee x_{vG} \vee y_{vG} \vee y_{vB}$. Now we show that any $\bar{x}_{vi}$ is a repair for an at-least-one clause $x_{v1} \vee x_{v2} \vee \dots \vee x_{vd}$. Literal $\bar{x}_{vi}$ occurs only in at-most-one clauses $\bar{x}_{vi} \vee \bar{x}_{vj}$, and $\bar{x}_{vj}$ is the negation of a literal in the at-least-one clause, so the at-least-one clause is NC-retractable with repair $\bar{x}_{vi}$.

3.2 Auxiliary variable definitions

Another SAT modelling technique is the use of auxiliary variables to express certain constraints. When defining auxiliary variable we sometimes have the choice of using an if- or an if-and-only-if style of definition. (A similar choice may occur when relating two models of a problem by channelling constraints.) For example we may have an auxiliary variable a definition $a \leftrightarrow \ell_3 \vee \ell_4$, or in clause form

$$\bar{a} \vee \ell_3 \vee \ell_4 \quad a \vee \bar{\ell}_3 \quad a \vee \bar{\ell}_4$$

Such a definition may arise if we wish to eliminate clauses with more than three literals, by replacing $\ell_1 \vee \ell_2 \vee \ell_3 \vee \ell_4$ by $\ell_1 \vee \ell_2 \vee a$. The clauses of the form $a \vee \bar{\ell}_i$ are NC-retractable because the literal $\bar{a}$ is a repair for each.

Another form of auxiliary variable definition is $a \leftrightarrow \ell_1 \wedge \ell_2$, or in clause form

$$\bar{a} \vee \ell_1 \quad \bar{a} \vee \ell_2 \quad a \vee \bar{\ell}_1 \vee \bar{\ell}_2$$

Such a definition may be used if two literals ℓ_1 and ℓ_2 often occur together. The clause $a \vee \bar{\ell}_1 \vee \bar{\ell}_2$ is NC-retractable because $\bar{a}$ is a repair.

Thus clause retraction can transform problems from an if-and-only-if-style to an if-style of auxiliary variable definition. The latter style is often adopted by problem modellers: we may be interested only in the truth assignments of the original problem variables, and can ignore the values assigned to the auxiliary variables.

3.3 Robustness under problem transformation

It is interesting to note that retraction is robust under problem transformations such as those described in [1]. These are satisfiability-preserving transformations of SAT problems such as: permute the variable names; permute the clause order; permute the literal order within clauses; systematically permute the signs of literals for randomly-chosen variables. These transformations have been applied in SAT solver competitions to improve their fairness. It can be verified that any form of retractability is preserved under these transformations.

4 Experiments

We evaluate the clause retraction technique on SAT benchmarks from the SATLIB library,¹ using the simple NC-retractability test: Δ and Δ' contain mutually negated literals and are therefore mutually inconsistent. For each problem we found best mean results over 1000 runs (or 100 for harder problems) for Walksat with default (SKC) heuristics [12], testing noise levels 0.1–0.9. The results fall into several categories:

- Some problems contain few or no retractable clauses and retraction makes little or no difference. Examples are random 3-SAT, many of the Beijing circuit synthesis problems (such as `2bitadd` and `2bitcomp`), some planning problems (`anomaly`, `medium`, `huge` and the Blocks World problems), the graph colouring problems (which are SAT-encoded without at-most-one clauses), the parity learning and quasigroup completion problems.
- Some problems are improved by retraction. For example the circuit synthesis problem `3blocks` has 49% retractable clauses, the number of flips is reduced by 12% and the time by 45%. `4blocks` has 54% retractable clauses, the number of flips is reduced by 12% and the time by 51%.
- Some problems are made worse by retraction, perhaps because useful implied clauses are retracted. For example `logistics.a` has 22% retractable clauses, the number of flips is increased by 14% and the CPU time by 6%. `logistics.b` has 21% retractable clauses, the number of flips is increased by 6% and the time by 1%.
- Some problems have a significant number of retractable clauses, but because they are hard for local search we have no systematic results as yet. For example the circuit synthesis problem `e0ddr2-10-by-5-1` has 16% retractable clauses, the Hanoi Towers problem `hanoi4` has 15% and `hanoi5` has 12%, the bounded model checking problem `bmc-ibm-1` has 5%, and the hardware verification problem `dlx2_cc_a_bug17` has 10%.
- Some contain a significant number of retractable clauses, and retracting them improves the flip rate while making no significant difference in the number of flips required to find a solution, thus improving the run time.

Problems that were improved by retraction had similar optimal noise levels for Walksat before and after retraction. In contrast, some that were made worse by retraction required more noise after retraction, perhaps indicating that new local minima have been created by the removal of implied clauses.

Clause retraction is fast even with the current prototype implementation, taking far less time than is required to solve the problems. These results are preliminary but promising. Better results might be obtained by a stronger test for retractability, for example using unit propagation to check satisfiability.

5 Related work

Given a SAT problem P , the approach of [7] is to generate a related problem P' that is (hopefully) easier to solve but which preserves satisfiability (P' is satisfiable if and

¹ <http://www.cs.ubc.ca/hoos/SATLIB/>

only if P is). They describe several transformations yielding P' with higher solution density than P .

Retractable constraints and clauses are closely related to the work of [2], who describe a technique for the *safe delay* of constraints in a high-level specification. Safe delay constraints can be temporarily removed from the specification before the problem is solved, and solutions can later be transformed to satisfy them. But our approach works directly on a CSP or SAT problem. Retractability can also be enhanced by exploiting higher forms of consistency. It is also robust under simple problem transformations, whereas safe delay constraints require (we believe) the specification to take a more specific form.

There is also a close connection with one aspect of the work of [5], who define conditions under which at-most-one and at-least-one clauses can be removed from a SAT-encoding of a CSP. As shown in Section 3.1 our method can retract these clauses under similar conditions, and we believe that their condition for removing the clauses is equivalent to NC-retraction, though this remains to be shown. If it is true then our method can remove more clauses as it can be made more powerful by using higher consistency than NC, and it is not restricted to at-least-one and at-most-one clauses.

Repairing pseudo-solutions to true solutions may be related to the robustness work of [6] and others, in which a *supermodel* is a CSP/SAT solution that can be *repaired* to another solution by making small changes, if the problem constraints/clauses change slightly. The applications are different, as we do not consider changes in a CSP or SAT problem or look for robust solutions. However, there may be techniques that can be adapted between the two areas.

6 Conclusion

We showed that a new automatic reformulation technique can remove conflicts or clauses from a constraint satisfaction or SAT problem, and that it generalises some known modelling techniques. Like most transformations, conflict/clause retraction is not guaranteed to improve a problem:

- It may not be beneficial for backtrack search, which usually benefits from the *addition* of constraints or clauses because they reduce the search space.
- Any transformation may combine poorly with search heuristics, and such effects are currently hard or impossible to predict.
- It may retract useful implied constraints or clauses.

However, in experiments on SAT benchmarks it gave promising results. This work can be developed in several ways:

- We will test unit-propagation-retraction in SAT and AC-retraction in CSPs.
- Retracting a SAT clause may create new pure literals whose clauses can then be deleted, which may in turn lead to further clause retraction.
- Retracting conflicts from a random problem might move it out of a phase transition, and make it easier to solve. NC-retraction did not appear to help random 3-SAT problems but unit-propagation-retraction might.

Acknowledgements This material is based in part upon works supported by the Science Foundation Ireland under Grant No. 00/PI.1/C075. Thanks to the anonymous referees for constructive comments.

References

1. F. Brglez, X. Y. Li, M. F. Stallman. The Role of a Skeptic Agent in Testing and Benchmarking of SAT Algorithms. *Fifth International Symposium on the Theory and Applications of Satisfiability Testing*, University of Cincinnati, 2002, pp. 354–361.
2. M. Cadoli, T. Mancini. Automatic Reformulation of Specifications by Safe Delay of Constraints. *Second International Workshop on Modelling and Reformulating Constraint Satisfaction Problems*, Kinsale, Cork, Ireland, 2003, pp. 33–47.
3. D. Clark, J. Frank, I. Gent, E. MacIntyre, N. Tomov, T. Walsh. Local Search and the Number of Solutions. *Proceedings of the Second International Conference on Principles and Practices of Constraint Programming, Lecture Notes in Computer Science* vol. 1118, Springer-Verlag 1996, pp. 119–133.
4. S. Colton, L. Drake, A. M. Frisch, I. Miguel, T. Walsh. Automatic Generation of Implied Constraints: Initial Progress. *Proceedings of the Eighth Workshop on Automated Reasoning*, York, 2001, pp. 17–18.
5. A. Frisch, T. Peugniez, A. Doggett, P. Nightingale. Solving Non-Boolean Satisfiability Problems with Stochastic Local Search: A Comparison of Encodings. *Journal of Automated Reasoning*, January 2006.
6. M. L. Ginsberg, A. J. Parkes, A. Roy. Supermodels and Robustness. *Fifteenth National Conference on Artificial Intelligence and Tenth Innovative Applications of Artificial Intelligence Conference*, 1998, pp. 334–339.
7. Y. Hanatani, T. Horiyama, K. Iwama. Density Condensation of Boolean Formulas. *Sixth International Conference on Theory and Applications of Satisfiability Testing*, 2003, pp. 126–133.
8. J. Marques-Silva. Algebraic Simplification Techniques for Propositional Satisfiability. *Sixth International Conference on Principles and Practice of Constraint Programming*, 2000.
9. S. Minton, M. D. Johnston, A. B. Philips, P. Laird. Minimizing Conflicts: A Heuristic Repair Method for Constraint Satisfaction and Scheduling Problems. *Artificial Intelligence* vol. 58 nos. 1–3, 1992, pp. 160–205.
10. S. D. Prestwich. Local Search on SAT-Encoded Colouring Problems. *Sixth International Conference on the Theory and Applications of Satisfiability Testing, Lecture Notes in Computer Science* vol. 2919, Springer, 2003, pp. 105–119.
11. S. D. Prestwich. Increasing Solution Density by Dominated Relaxation. *Fourth International Workshop on Modelling and Reformulating Constraint Satisfaction Problems*, Sitges, Spain, 2005, pp. 1–13.
12. B. Selman, H. Kautz, B. Cohen. Noise Strategies for Improving Local Search. *Twelfth National Conference on Artificial Intelligence*, AAAI Press, 1994, pp. 337–343.
13. B. Selman, H. Levesque, D. Mitchell. A New Method for Solving Hard Satisfiability Problems. *Tenth National Conference on Artificial Intelligence*, MIT Press, 1992, pp. 440–446.
14. T. Walsh. SAT v CSP. *Sixth International Conference on Principles and Practice of Constraint Programming, Lecture Notes in Computer Science* vol. 1894, Springer-Verlag, 2000, pp. 441–456.